



# Compiler Design

---

**Preet Kanwal**

Department of Computer Science & Engineering

Teaching Assistant : Kavya P K

# Compiler Design

---

## Unit 4: Live-Variable Analysis

**Preet Kanwal**

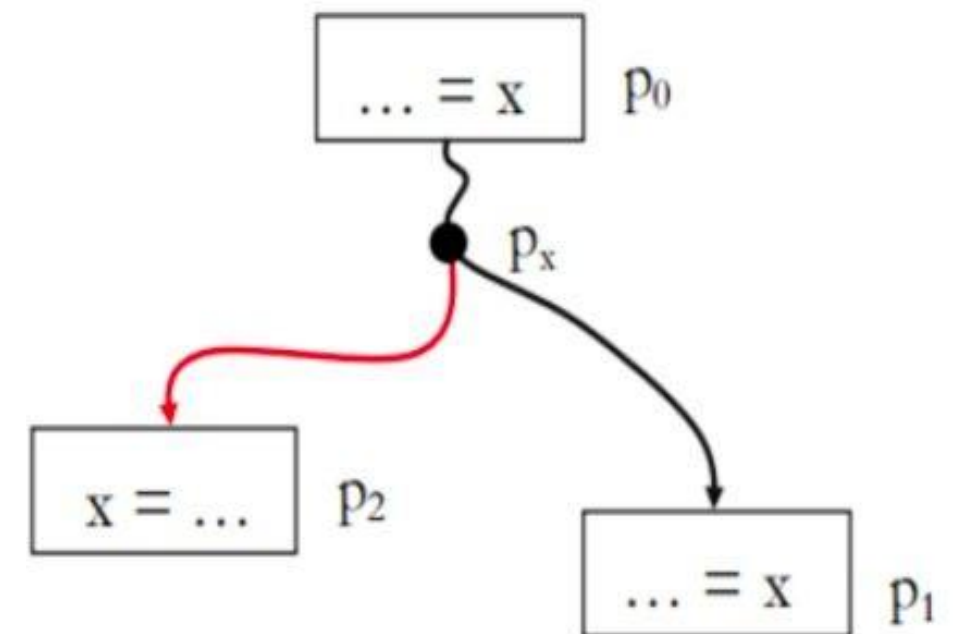
Department of Computer Science & Engineering

In this lecture, you will learn about -

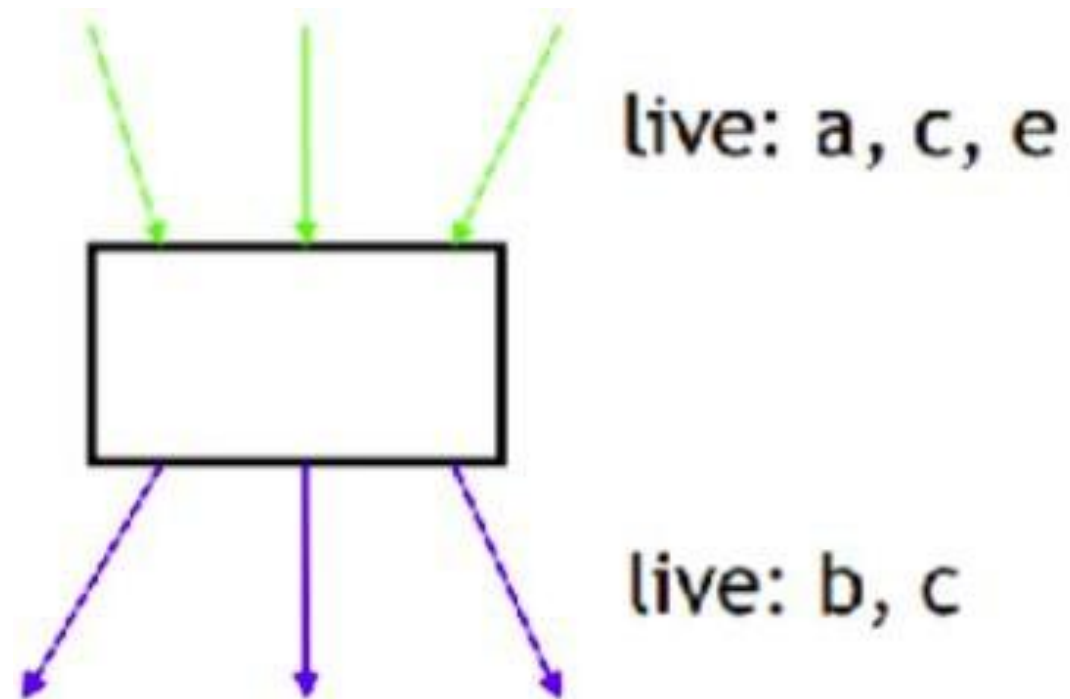
- What is Live variable analysis?
- Liveness
- Applications
- Data flow values and equations
- Live-Variable Analysis Algorithm
- Examples

## What is Live-Variable Analysis?

- Live variable analysis is based on the following definition -
  - For a variable **x** and program point **p** determine if the value of **x** at **p** can still be used along some path starting at **p**.
  - If yes, **x** is live at **p**.
  - If not, **x** is dead at **p**.
- In the given illustration -
  - At point **p<sub>0</sub>** the **x** variable is live if there is a path from **p<sub>0</sub>** where **x** is used.
  - Beyond **p<sub>x</sub>** towards **p<sub>2</sub>** the value of **x** is no longer needed and is dead.



- Liveness is associated with edges of control flow graph, not nodes (statements).



- **Register Allocation** - If a variable is dead at a given point **p**, we can reuse its storage, i.e, the register it occupies (if any).
- **Dead code elimination** - If variable **x** is not live after an assignment **x = ...**, then the assignment is redundant and can be deleted as dead code.

Live variable Analysis algorithm requires the computation of the following data-flow values -

- **use[n]** : set of variables used by Node **n**
- **def [n]** : set of variables defined by Node **n**
- **in[n]** : variables live on entry to Node **n**
- **out[n]** : variables live on exit from Node **n**



The data-flow values are then used to compute IN and OUT sets for each basic block B using the data flow equations-

$$\text{OUT [B]} = \bigcup_{S \text{ is a successor of B}} (\text{IN}[S])$$

$$\text{IN [B]} = \text{use[B]} \cup (\text{OUT[B]} - \text{def[B]})$$



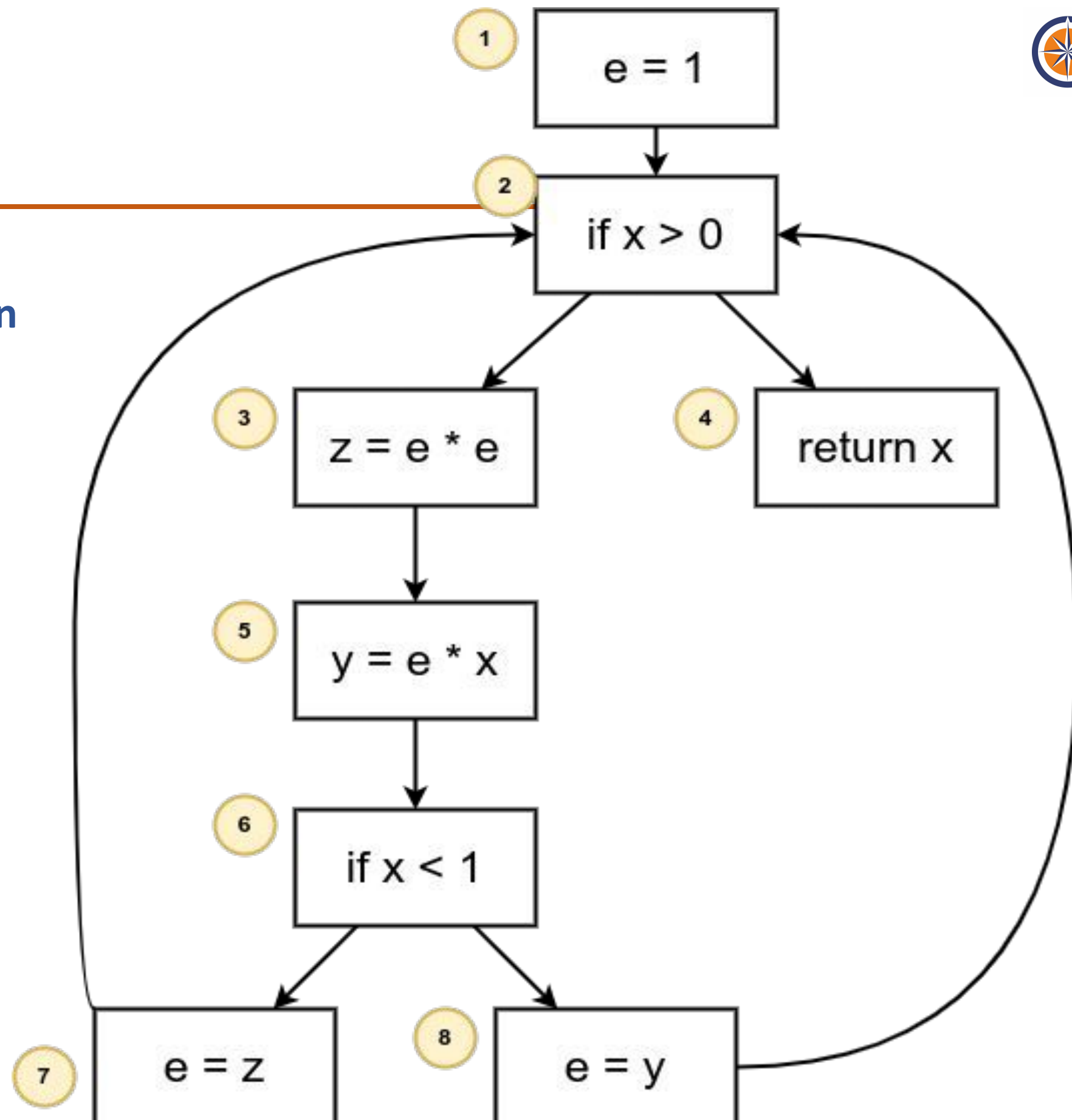
## Live-Variable Analysis Algorithm

---

### Algorithm

```
for each node  $n$  in CFG  
     $\text{in}[n] = \emptyset$ ;  $\text{out}[n] = \emptyset$  } Initialize solutions  
repeat  
    for each node  $n$  in CFG in reverse topsort order  
         $\text{in}'[n] = \text{in}[n]$   
         $\text{out}'[n] = \text{out}[n]$  } Save current results  
         $\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$   
         $\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$  } Solve data-flow equations  
until  $\text{in}'[n] = \text{in}[n]$  and  $\text{out}'[n] = \text{out}[n]$  for all  $n$  } Test for convergence
```

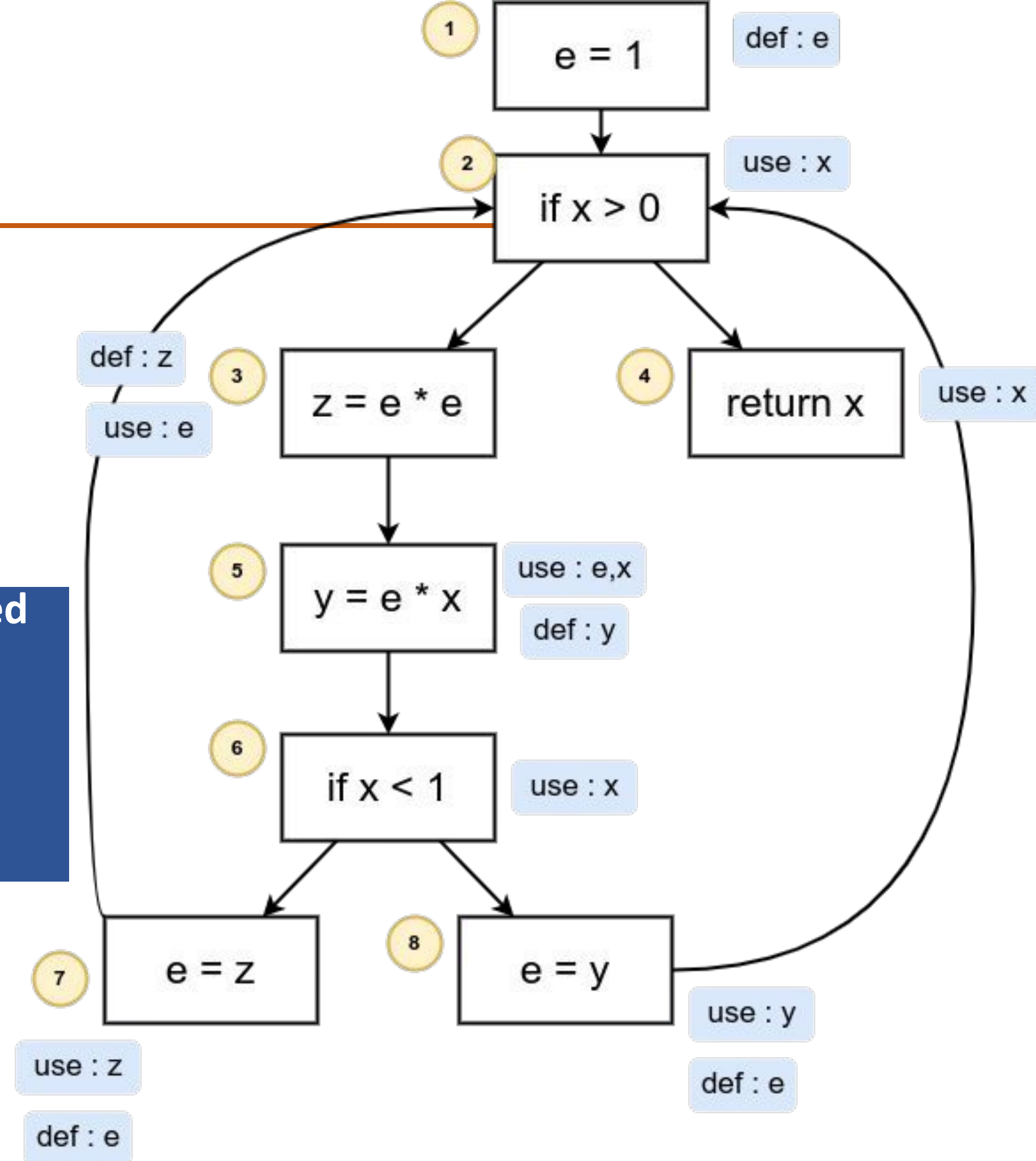
Compute Liveness Information  
for the following CFG -



Step 2 - Compute use and def for each basic block.

**use[n]** : set of variables used by Node **n**

**def [n]** : set of variables defined by Node **n**

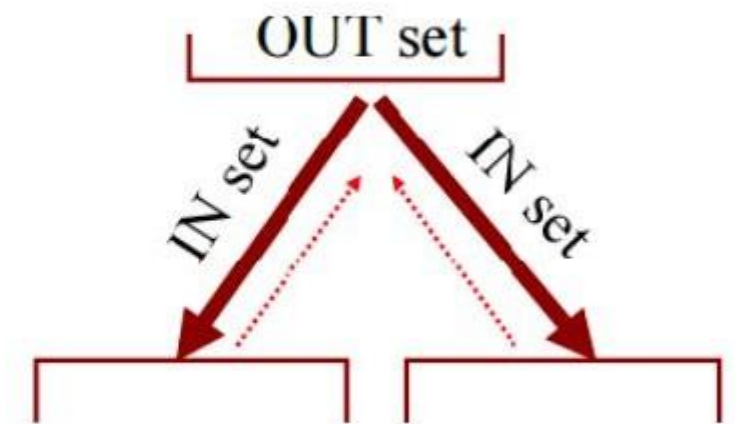


### Step 3 - Compute IN and OUT for each basic block

$$OUT [B] = \bigcup_{S \text{ is a successor of } B} ( IN[S] )$$

$$IN [B] = use[B] \cup ( OUT[B] - def[B] )$$

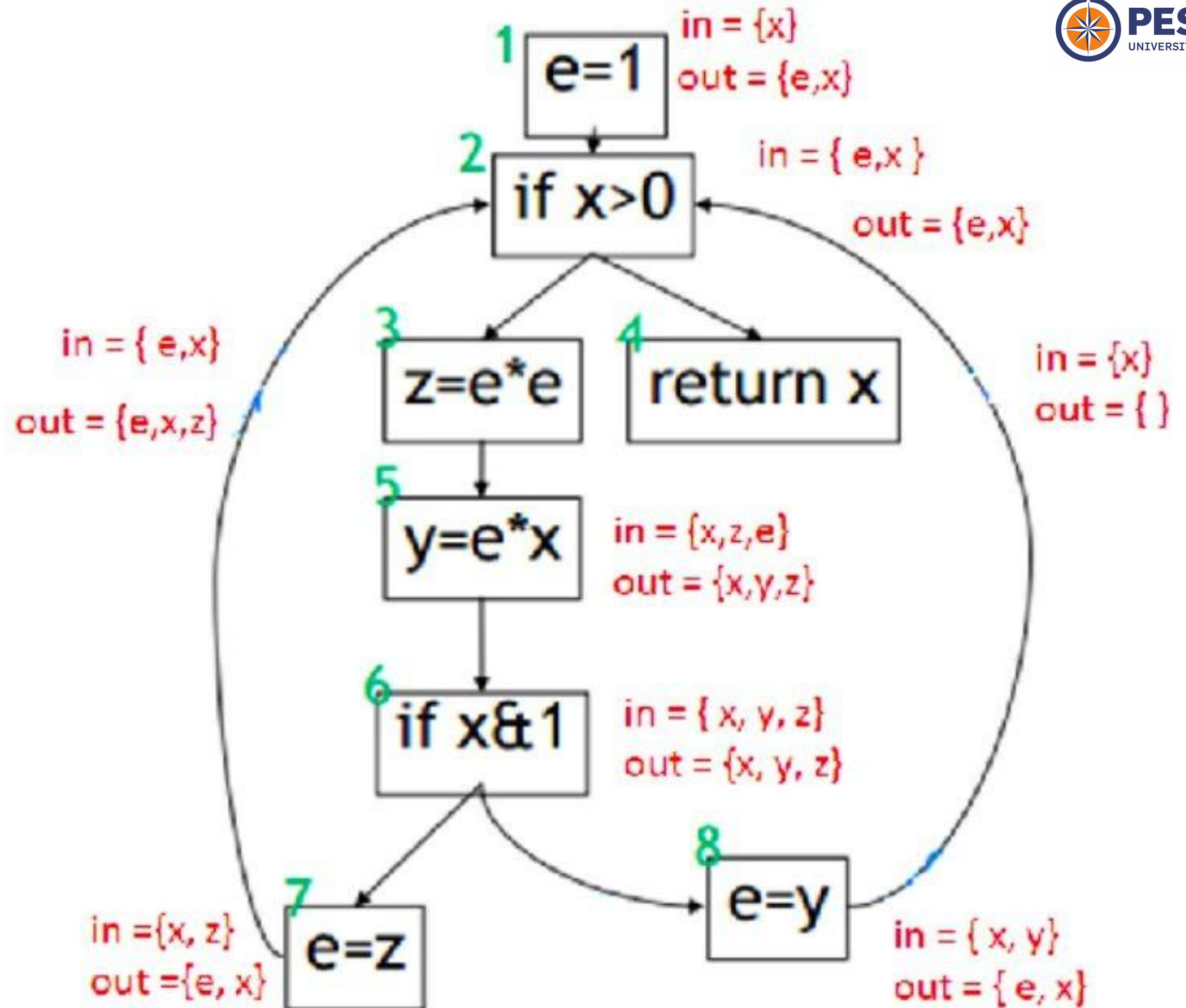
$$OUT[B] = \bigcup_{S \text{ is a successor of } B} IN[S]$$



$$IN(B) = Use(B) \cup (OUT(B) - Def(B))$$

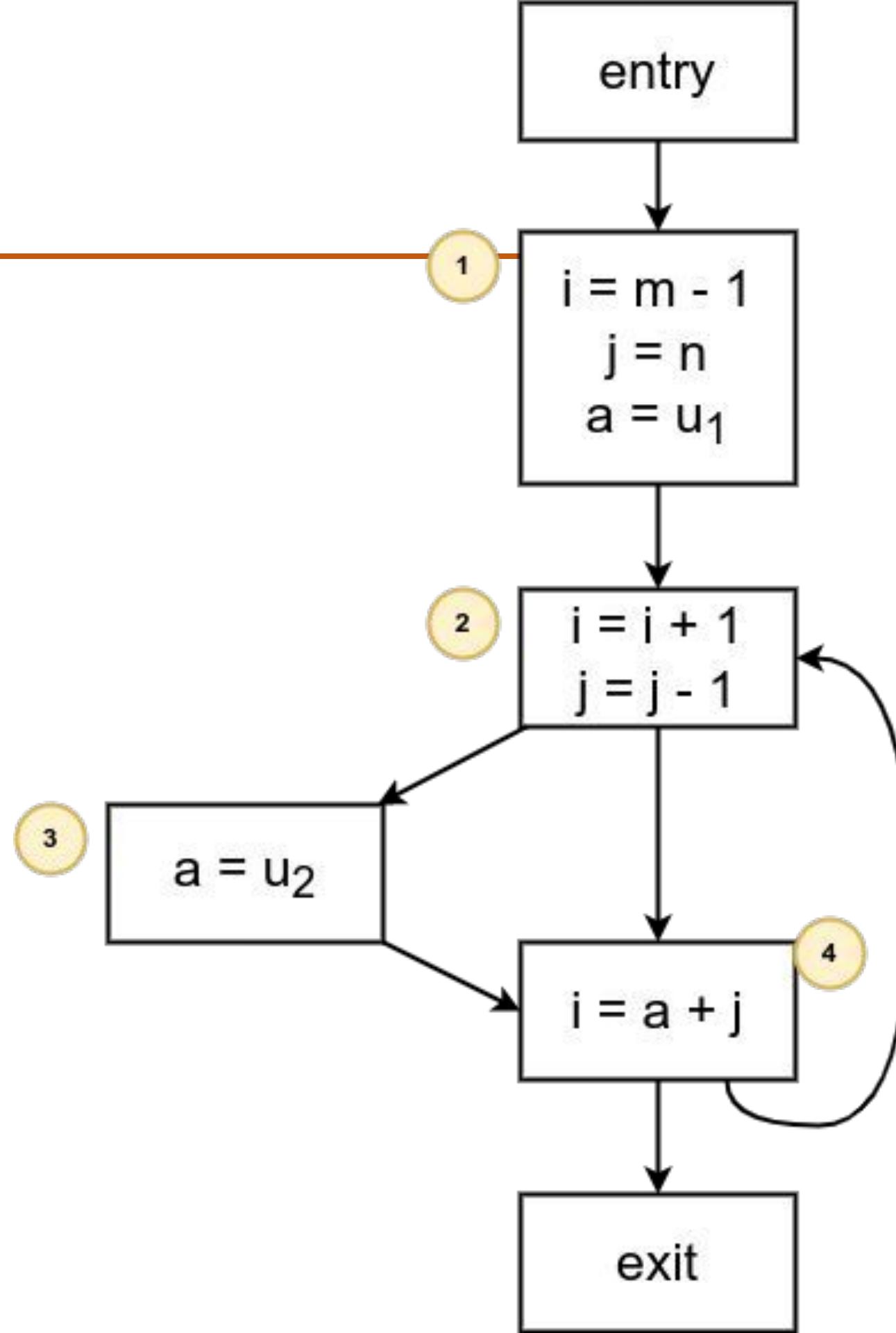
# Compiler Design

## Example 1 Solution

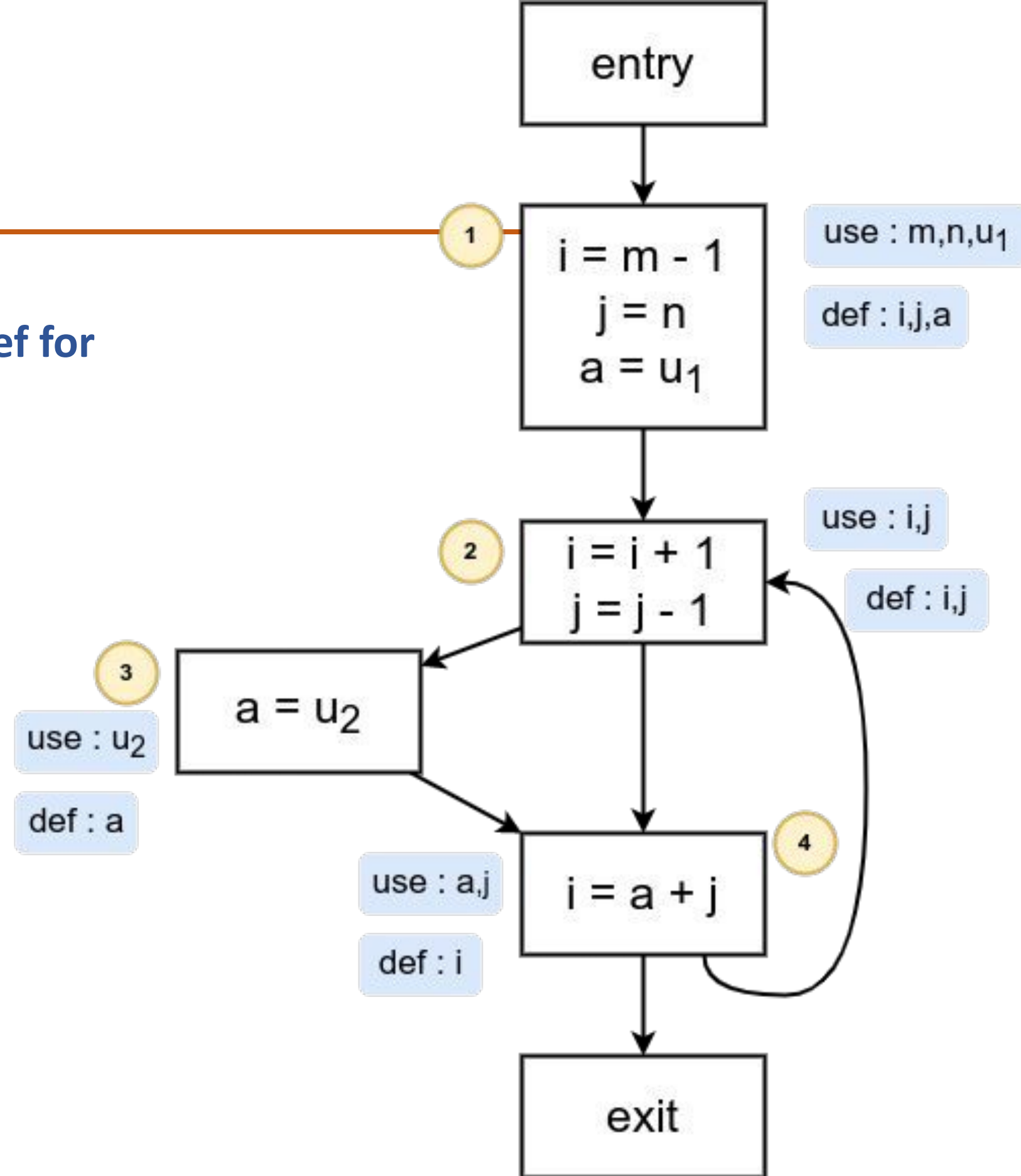




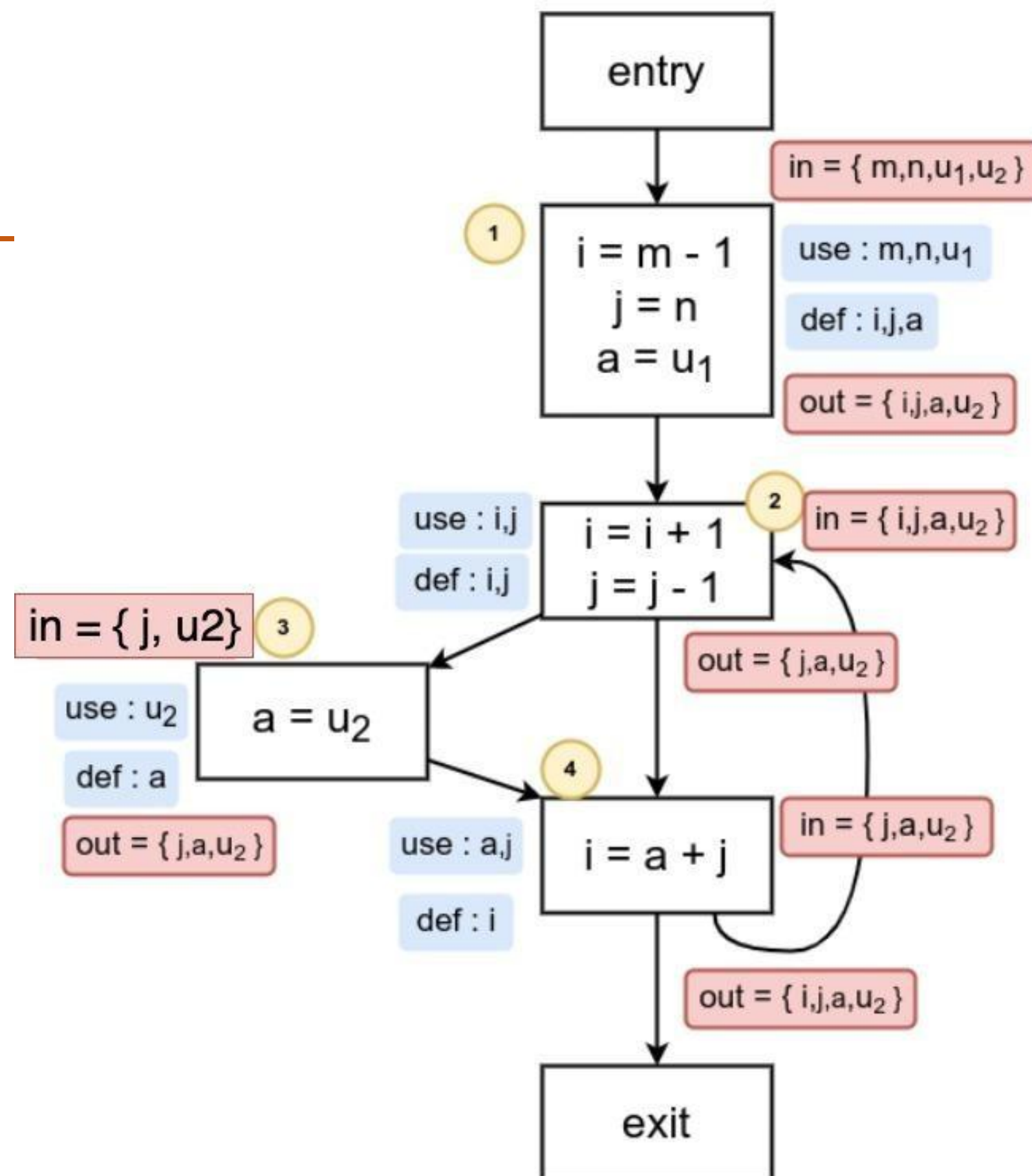
Compute Liveness Information  
for the following CFG -



Step 2 - Compute use and def for each basic block.

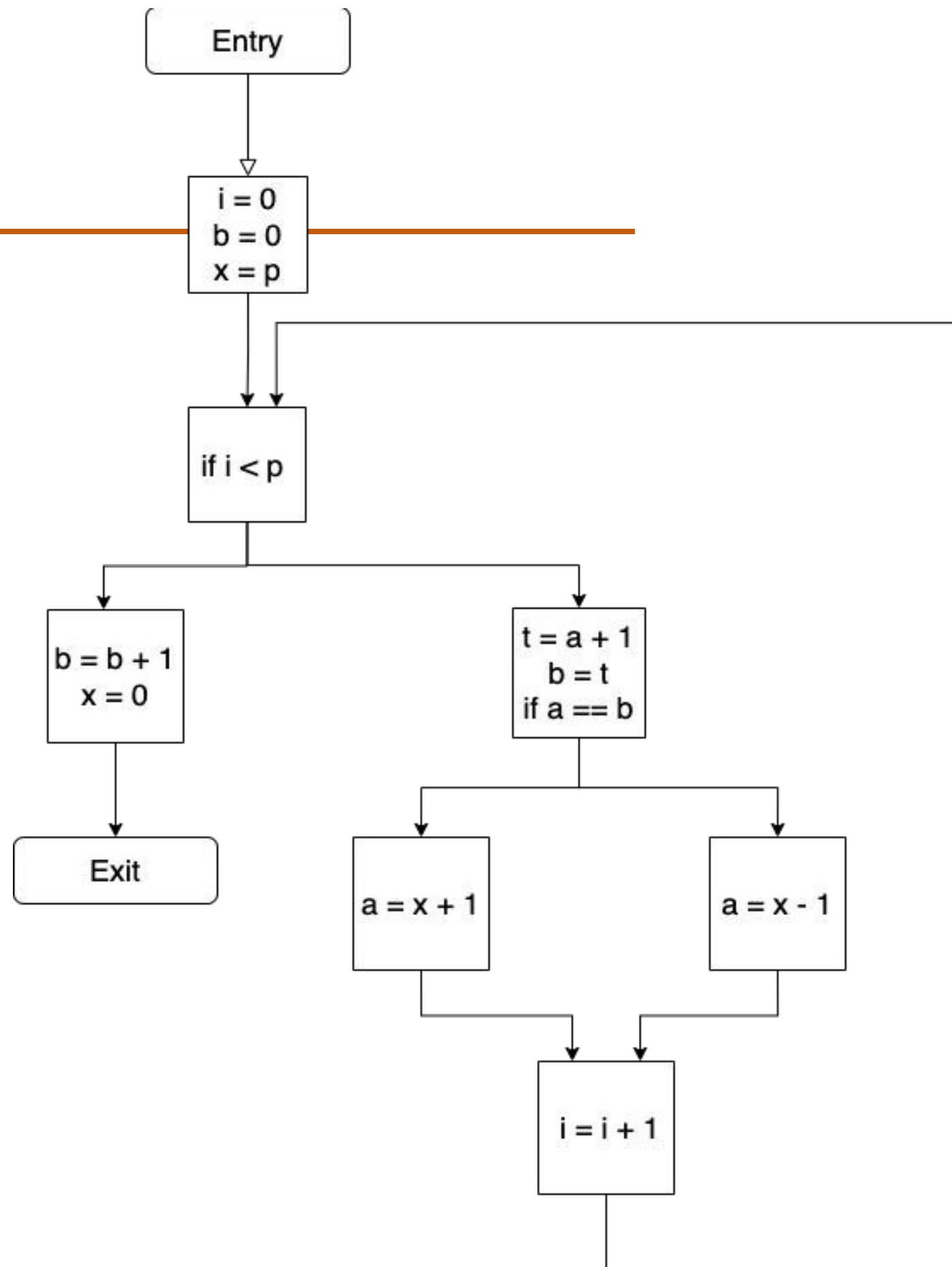


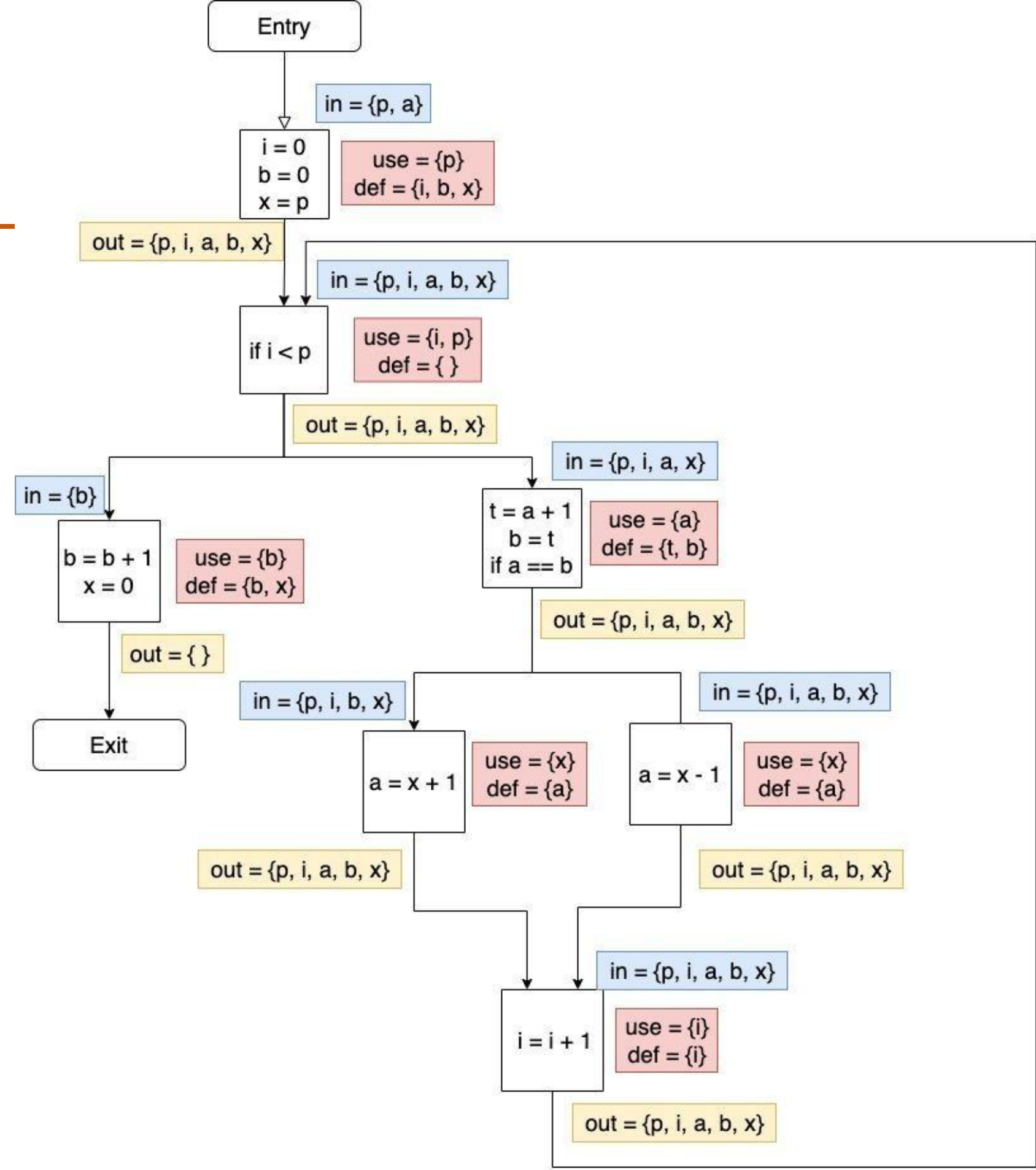
Step 3 - Compute IN and OUT for each basic block



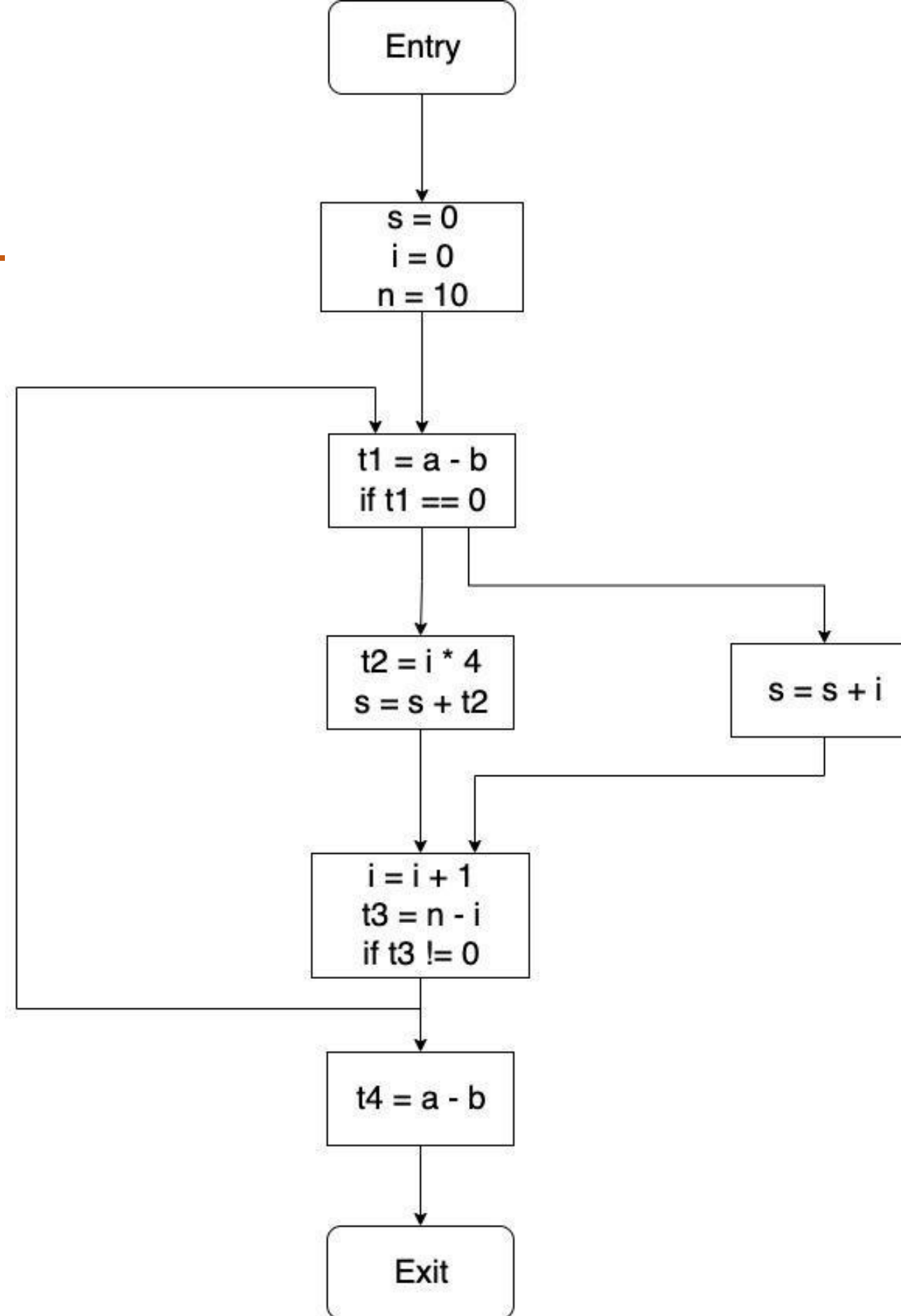


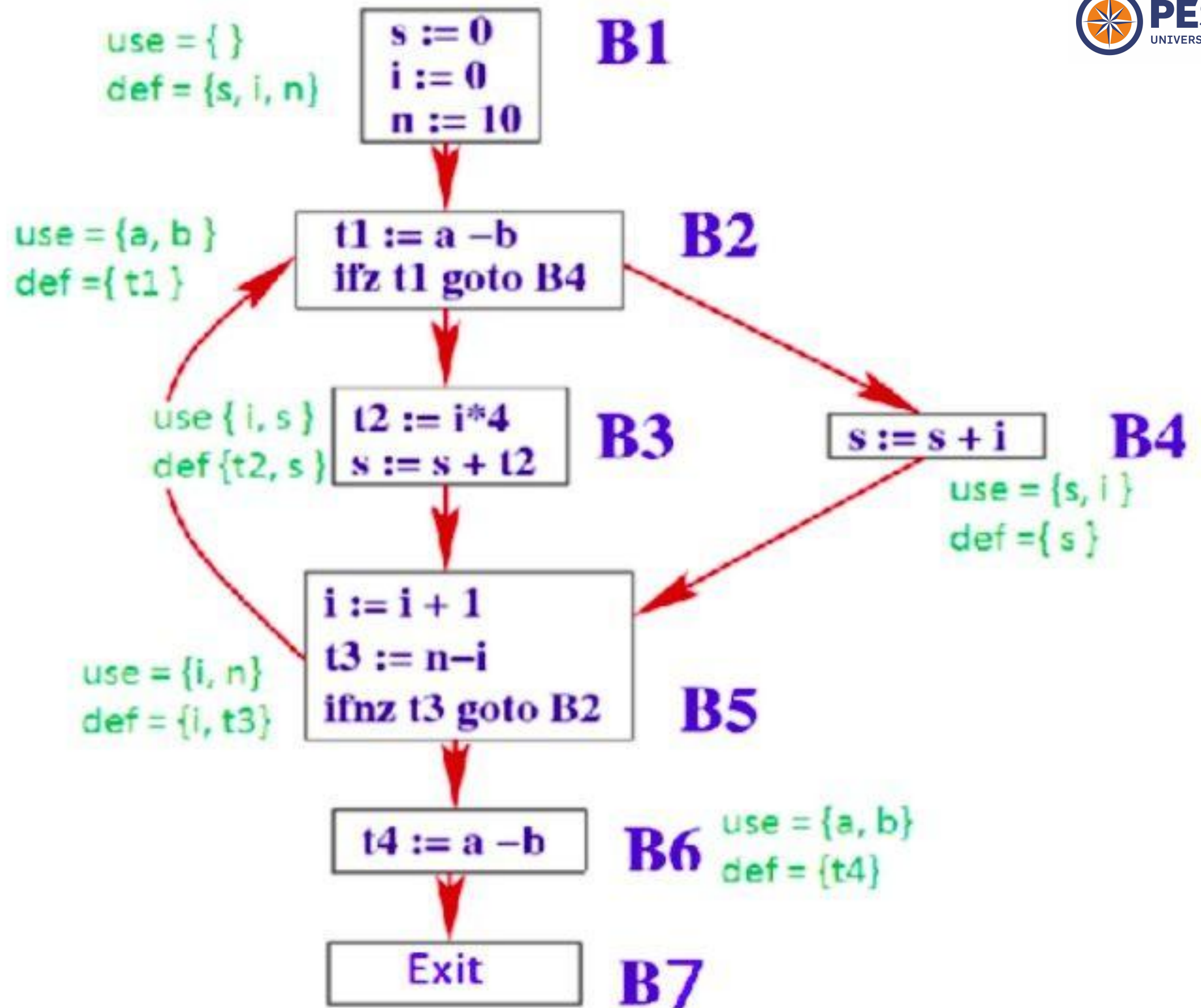
Compute Liveness Information  
for the following CFG -

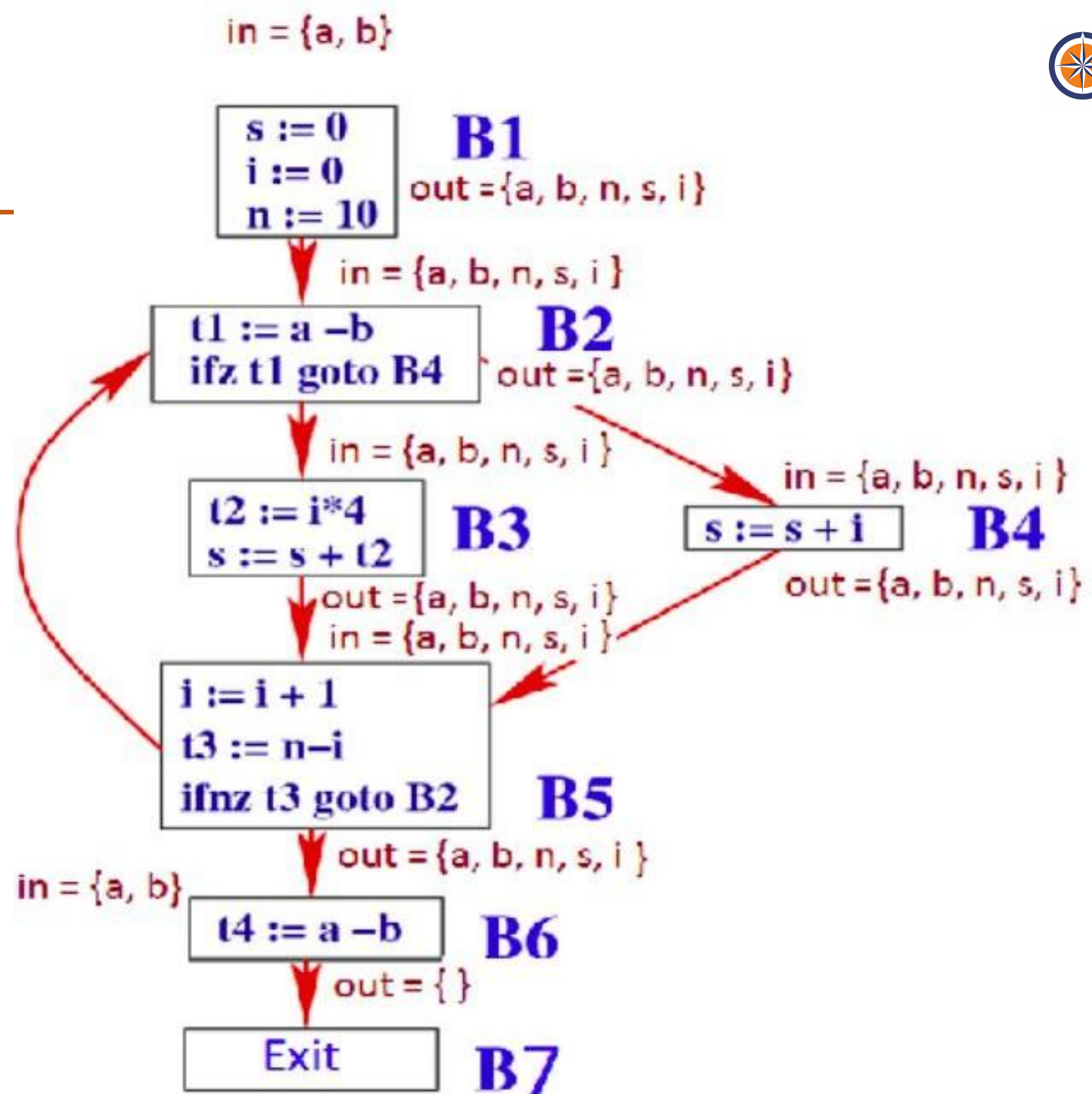




Compute Liveness Information  
for the following CFG -

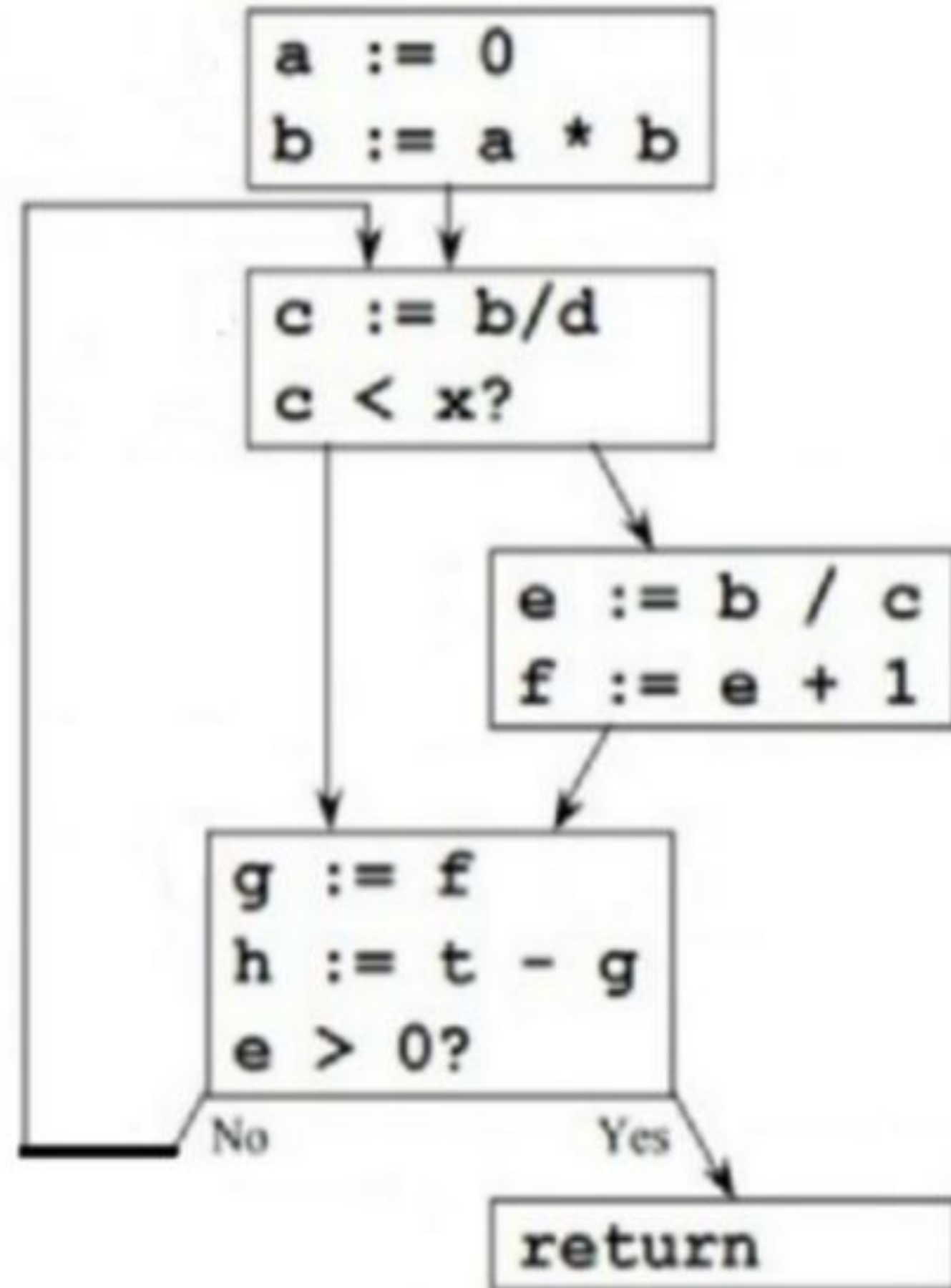




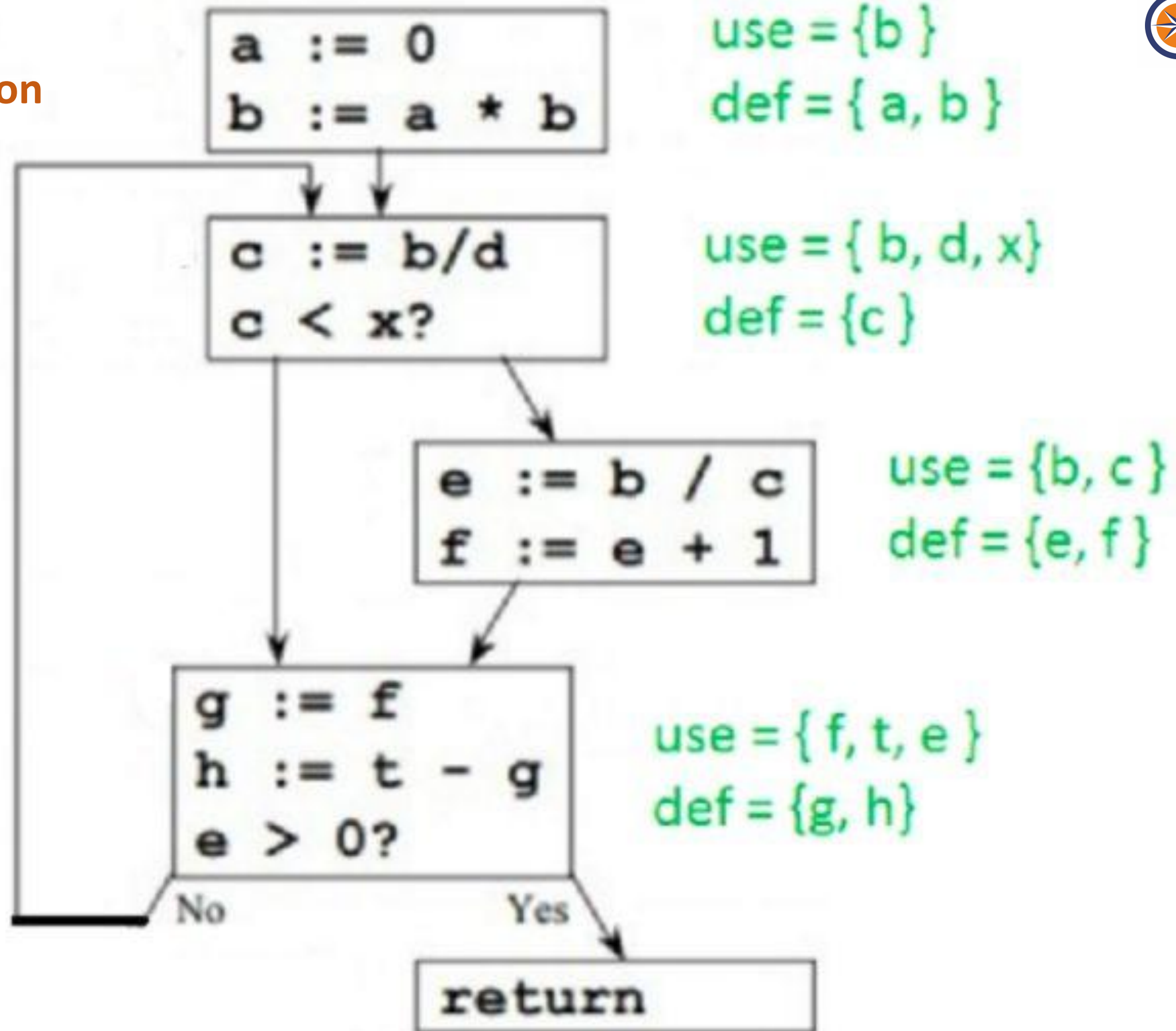


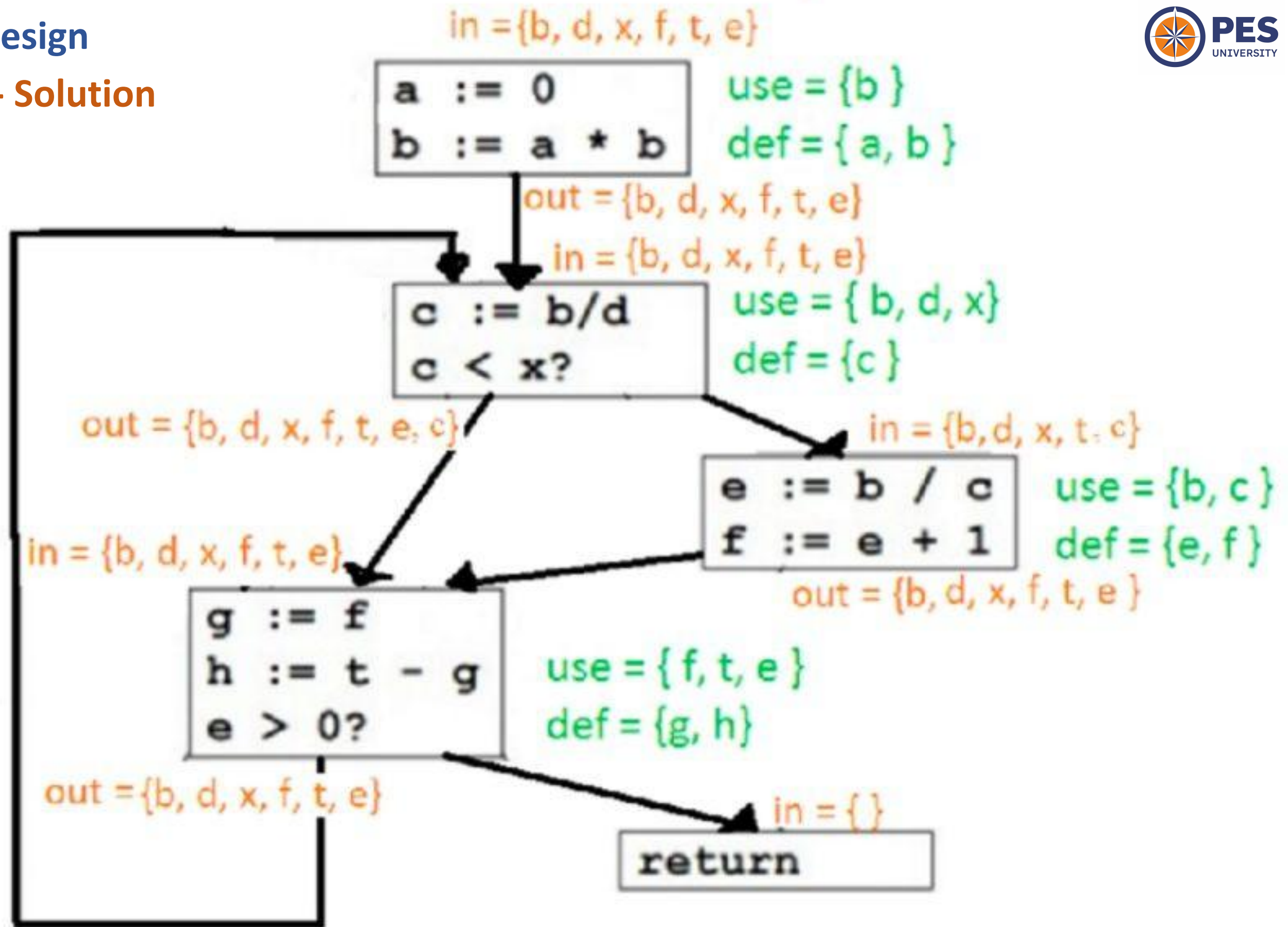


Compute Liveness  
Information for the  
following CFG -



Step 2 - Compute use and def for each basic block.









**THANK  
YOU**

---

**Preet Kanwal**

Department of Computer Science & Engineering

[preetkanwal@pes.edu](mailto:preetkanwal@pes.edu)